Advanced Networking Protocol Implementation and Security

Python

Pilusa Obed
EDUVOS Pretoria Campus

Table of Contents

Secure Data Transmission via TLS 2

Server code: 2

Client Code: 3

The importance of using TLS 5

Mutual Authentication 5

Advanced DNS Resolution Strategies 7

Security Extensions (DNSSEC): 8

Parallel Querying: 9

Privacy via DoH: 10

Optimized Performance: 11

Concurrent TCP File Management: 14

Concurrency 14

Reliability: 16

Real-time UDP Communication 17

Client Management 17

Asynchronous Messaging 19

Works Cited 21

Secure Data Transmission via TLS

Transport Layer Security (TLS) is a transport layer protocol derived from Secure Socket Layer (SSL) that secures data in transit (GeeksforGeeks, 2024). I used OpenSSL to generate the server's authentication certificate with the help of (Bertram, 2020)

```
C:\Users\pilus\OneDrive\Documents\School Projects>openssl genrsa -o
ut server.key 2048

C:\Users\pilus\OneDrive\Documents\School Projects>openssl req -new
-key server.key -out server.csr
You are about to be asked to enter information that will be incorpo
rated
into your certificate request.
What you are about to enter is what is called a Distinguished Name
or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:South Africa
String too long, must be at most 2 bytes long
Country Name (2 letter code) [AU]:SA
State or Province Name (full name) [Some-State]:Gauteng
Locality Name (eg, city) []:Pretoria
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Eduvos
Organizational Unit Name (eg, section) []:Project
Common Name (e.g. server FQDN or YOUR name) []:server.csr
Email Address []:eduv4910561@vossie.net

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:Secure1234
An optional company name []:Project Q1
```

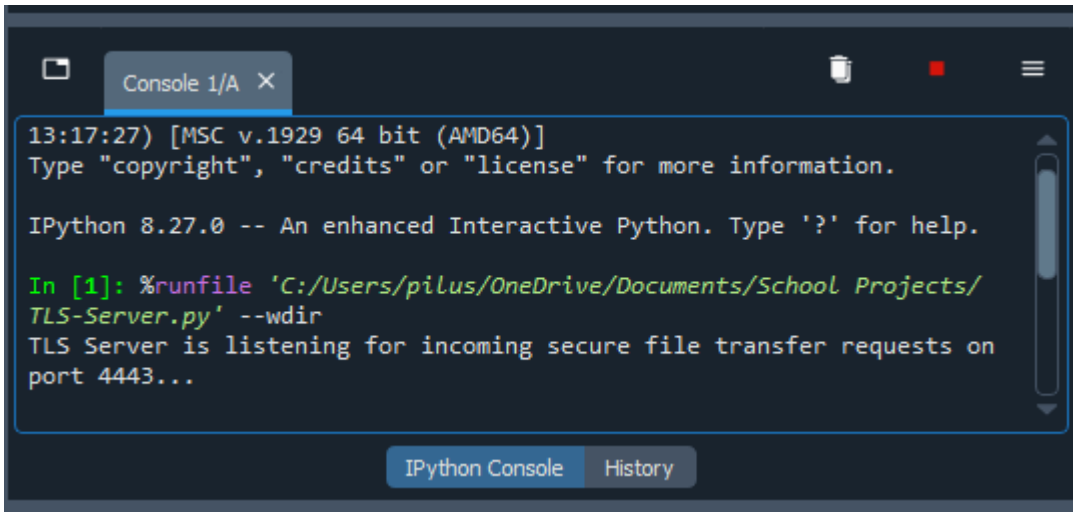
Server code:

Below you'll find the TLS server script:

```
import ssl  # Secure Sockets Layer (SSL) for encrypted communication

import socket  # Allows creation of network sockets


# Define server address and port

server_address = ('0.0.0.0', 4443)  # Listens on all available interfaces


# Create a standard TCP socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)


# Wrap the socket with TLS
```

```
tls_context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
tls_context.load_cert_chain(certfile='server.crt', keyfile='server.key')  # Load certificate and private key

secure_socket = tls_context.wrap_socket(server_socket, server_side=True)  # Wrap socket with TLS

# Bind and listen for connections
secure_socket.bind(server_address)
secure_socket.listen(5)  # Allow up to 5 clients in the connection queue
print("TLS Server is listening for incoming secure file transfer requests on port 4443...")

# Keep the server running to listen for incoming connections
try:
    while True:
        pass  # The server is listening
except KeyboardInterrupt:
    print("\nShutting down the server.")
finally:
    secure_socket.close()  # Close the socket when the server is stopped
```

Server Results:



Client Code:

Below you'll find the client code that sends a file "Financial_Reports" securely using TLS:

```
import ssl
import socket
```

```
# Define server address and port
server_address = ('localhost', 4443)  # Connects to the local TLS server

# Create a TCP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Set up the TLS context
tls_context = ssl.create_default_context()
tls_context.load_verify_locations(cafile='server.crt')  # Trust the server's certificate
tls_context.check_hostname = True  # Verify the server hostname matches certificate's CN

# Wrap the socket with TLS
secure_socket = tls_context.wrap_socket(client_socket, server_hostname='server.csr')

# Function to send a file to the server
def send_file(file_path):
    try:
        secure_socket.connect(server_address)  # Establish connection
        print("Connected to server.")

        # Open the file in binary mode and send content
        with open(file_path, 'rb') as file:
            print(f"Sending file: {file_path}")
            secure_socket.sendfile(file)

        # Receive server response
        response = secure_socket.recv(1024)
        print("Server response:", response.decode())

    except Exception as e:
        print("An error occurred:", e)

    finally:
        secure_socket.close()
        print("Connection closed.")

send_file('Finacial_Report.txt')  # Calls the function to send a file
```

Output:

```
C:\WINDOWS\system32\cmd.

C:\Users\pilus\OneDrive\Documents\School Projects>python TLS_Client.py
Connected to server.
Sending file: Finacial_Report.txt
An error occurred: [WinError 10054] An existing connection was forcibly closed by th
e remote host
Connection closed.
```

The importance of using TLS:

a. **Encryption** –TLS provides encryption for data transmitted over a network, which is crucial for dealing with confidential data such as financial reports (Joshua, 2023) .
b. **Authentication** – It Ensures that client and server identities are verified to be the intended sender and recipient, this ensures that the confidential data is accessed by authorized individuals (Metzler, 2024)
c. **Data Integrity** – It prevents modification of transmitted data by creating a digital signature from the encrypted data which verifies the integrity of the data and will ensure that the financial reports are not tempered with

Mutual Authentication

Modified server scrip to implement mutual authentication. Mutual TLS is an end-to-end authentication where the server and client verify their identities before establishing a connection (Ranke, 2023). Generated the client certificate using OpenSSL.

```
Command Prompt

C:\Users\pilus\OneDrive\Documents\School Projects>openssl genrsa -o
ut client.key 2048

C:\Users\pilus\OneDrive\Documents\School Projects>openssl req -new
-key client.key -out client.csr
You are about to be asked to enter information that will be incorpo
rated
into your certificate request.
What you are about to enter is what is called a Distinguished Name
or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:SA
State or Province Name (full name) [Some-State]:Gauteng
Locality Name (eg, city) []:Sunnyside
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Eduvos
Organizational Unit Name (eg, section) []:Project Q1.4
Common Name (e.g. server FQDN or YOUR name) []:Client01
Email Address []:Eduv4910561@vossie.net

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:Secure123@
An optional company name []:

C:\Users\pilus\OneDrive\Documents\School Projects>openssl x509 -req
 -days 365 -in client.csr -CA server.crt -CAkey server.key -set_ser
ial 01 -out client.crt
Certificate request self-signature ok
subject=C=SA, ST=Gauteng , L=Sunnyside, O=Eduvos, OU=Project Q1.4,
CN=Client01, emailAddress=Eduv4910561@vossie.net
```

Script:

```
import ssl
import socket

server_address = ('localhost', 4443)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
# Wraps the socket with TLS and enforces mutual authentication
tls_context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
tls_context.load_cert_chain(certfile='server.crt', keyfile='server.key')
tls_context.load_cert_chain(certfile='client.crt', keyfile='client.key')  # Require client authentication
tls_context.verify_mode = ssl.CERT_REQUIRED

secure_socket = tls_context.wrap_socket(server_socket, server_side=True)
secure_socket.bind(server_address)
secure_socket.listen(5)
print("TLS Server with mutual authentication listening on port 4443...")

while True:
    client_socket, client_address = secure_socket.accept()
    print(f"Connection established with {client_address}")

    data = client_socket.recv(1024)
    print("Received data:", data.decode())

    client_socket.send(b"File transfer received securely with mutual authentication.")

    client_socket.close()
```

Output with:

```
C:\Users\pilus\OneDrive\Documents\School Projects>
python TLS-Server.py
TLS Server is listening for incoming secure file t
ransfers on port 4443...
```

Advanced DNS Resolution Strategies

A Python Script that queries multiple DNS server in parallel and selects the fastest response, with the help from (Atkin, 2022) to understand multi-threading. The below scripts were completed with the help of (Bob Halley, 2021)

Installed dnspython requests

```
C:\Users\pilus>pip install dnspython requests
Defaulting to user installation because normal site-packages is not writeable
Collecting dnspython
  Downloading dnspython-2.7.0-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: requests in c:\users\pilus\appdata\local\packages\pythonsoftwarefoundation.python.3.13_q
z5n2kfra8p0\localcache\local-packages\python313\site-packages (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\pilus\appdata\local\packages\pythonsoftwarefoundati
n.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from requests) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in c:\users\pilus\appdata\local\packages\pythonsoftwarefoundation.python.3.
3_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\pilus\appdata\local\packages\pythonsoftwarefoundation.pyt
on.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from requests) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\pilus\appdata\local\packages\pythonsoftwarefoundation.pyt
on.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from requests) (2025.1.31)
Downloading dnspython-2.7.0-py3-none-any.whl (313 kB)
Installing collected packages: dnspython
Successfully installed dnspython-2.7.0
```

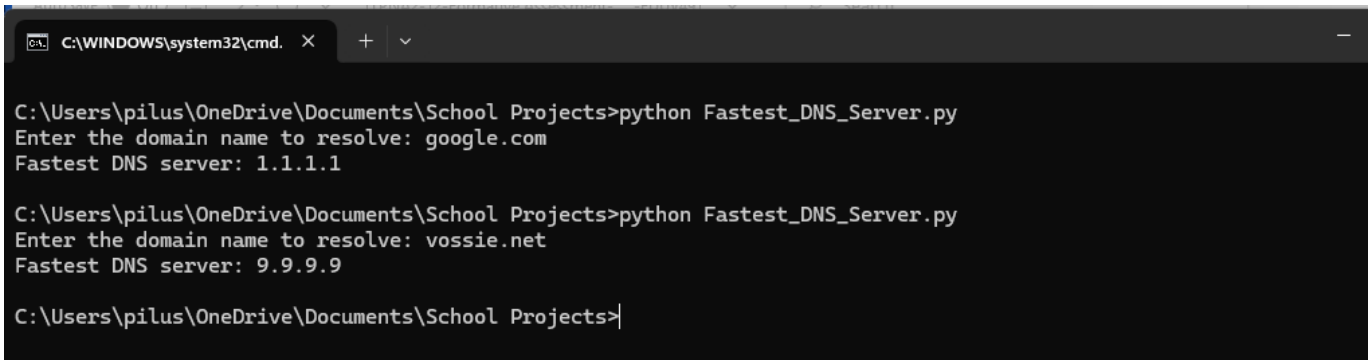
An exploration of performance and security enhancements for the Domain Name System.

Script:

```
import dns.resolver  # Used for DNS lookups
import threading  # Allows running tasks in parallel
import time  # Measures time taken for responses

servers = ['8.8.8.8', '1.1.1.1', '9.9.9.9']  # List of DNS servers to query
responses = {}  # Stores query times

# Function to query a DNS server
def query_dns(server, domain):
    start = time.time()  # Start timing
    resolver = dns.resolver.Resolver()  # Create a resolver object
    resolver.nameservers = [server]  # Use the specified DNS server
    try:
        resolver.resolve(domain, 'A')  # Query A record (IPv4 address)
        responses[server] = time.time() - start  # Store time taken
    except:
        responses[server] = float('inf')  # Assign high time if query fails

# Ask user for the domain name
domain = input("Enter the domain name to resolve: ").strip()

# Create and start threads for each DNS server
threads = [threading.Thread(target=query_dns, args=(server, domain)) for server in servers]
for thread in threads:
    thread.start()
for thread in threads:
    thread.join()
```

```
    fastest = min(responses, key=responses.get)  # Find the fastest server
    print(f"Fastest DNS server: {fastest}")
```

output:



Security Extensions (DNSSEC): Implementation of digital signatures to verify the integrity of DNS records.

DNSSEC **:** is a DNS extension protocol used for securing DNS records by utilizing digital signatures and cryptographic keys (GeeksforGeeks, 2024).

Script:

```
import dns.resolver


def secure_dns_query(domain):
    try:
        resolver = dns.resolver.Resolver()
        resolver.use_edns(edns=True, ednsflags=dns.flags.DO)  # Enable DNSSEC
        answer = resolver.resolve(domain, 'A')


        if answer.rrset is not None:
            print(f"DNSSEC validation successful for {domain}")
            for record in answer:
                print(f"Resolved IP: {record.to_text()}")
        else:
            print(f"DNSSEC validation failed for {domain}")
    except dns.resolver.NoAnswer:
        print("No DNSSEC response received.")
    except dns.resolver.NXDOMAIN:
```

```
        print("Domain does not exist.")
    except Exception as e:
        print(f"An error occurred: {e}")


# Example usage
domain = input("Enter the domain to resolve with DNSSEC: ").strip()
secure_dns_query(domain)
```

Output:

```
C:\Users\pilus\OneDrive\Documents\School Projects>python DNSSEC.py
Enter the domain to resolve with DNSSEC: google.com
DNSSEC validation successful for google.com
Resolved IP: 142.251.47.174

C:\Users\pilus\OneDrive\Documents\School Projects>
```

Parallel Querying: A multi-threaded script that queries multiple DNS servers (e.g., Google, Cloudflare) simultaneously to select the fastest response

DNS Load Balancing: is a technique used to spread DNS network traffic between multiple servers in order to improve server performance and availability (GeekstoGeeks, 2023). Below you will find the load balancing script

Script:

```
import dns.resolver
import threading


servers = ['8.8.8.8', '1.1.1.1', '9.9.9.9']  # List of DNS servers
server_load = {server: 0 for server in servers}  # Tracks load


def query_dns_with_load_balancing(domain):
    def query_dns(server):
        try:
            resolver = dns.resolver.Resolver()
            resolver.nameservers = [server]
            answer = resolver.resolve(domain, 'A')
            for record in answer:
```

```
            print(f"{server} resolved IP: {record.to_text()}")
        except Exception as e:
            print(f"{server} failed to resolve {domain}: {e}")

    # Select least-loaded server
    chosen_server = min(server_load, key=server_load.get)
    print(f"Load-balancing selected resolver: {chosen_server}")
    server_load[chosen_server] += 1

    # Perform query
    query_dns(chosen_server)

# Example usage
domain = input("Enter the domain to resolve with load balancing: ").strip()
query_dns_with_load_balancing(domain)
```

Output:

```
C:\Users\pilus\OneDrive\Documents\School Projects>python Load_Balancer.py
Enter the domain to resolve with load balancing: google.com
Load-balancing selected resolver: 8.8.8.8
8.8.8.8 resolved IP: 142.251.47.206
```

Privacy via DoH: Developed a DNS-over-HTTPS client to prevent eavesdropping by encrypting DNS traffic within HTTPS

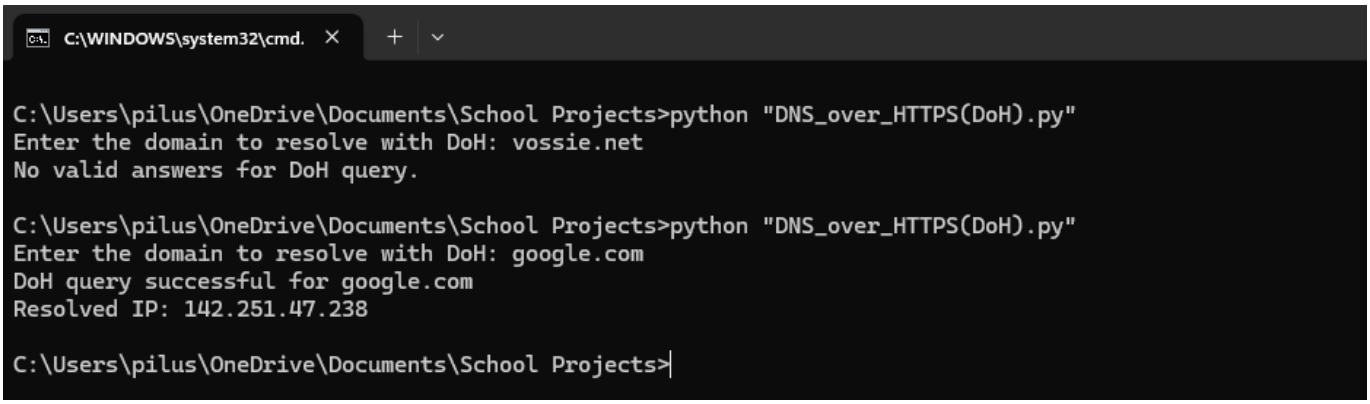
DNS-over-HTTPS (DoH): is a technique used to secure DNS traffic by encrypting it with HTTPS protocol (Schrader, 2025). The script below was crafted with the help of (forbiddenlake, 2023)

Script:

```
import requests

def query_doh(domain):
    doh_endpoint = "https://cloudflare-dns.com/dns-query"
    headers = {"accept": "application/dns-json"}
    params = {"name": domain, "type": "A"}
```

```
    try:
        response = requests.get(doh_endpoint, headers=headers, params=params)
        response.raise_for_status()
        data = response.json()

        if "Answer" in data:
            print(f"DoH query successful for {domain}")
            for answer in data["Answer"]:
                print(f"Resolved IP: {answer['data']}")
        else:
            print(f"No valid answers for DoH query.")
    except Exception as e:
        print(f"An error occurred querying DoH endpoint: {e}")

# Example usage
domain = input("Enter the domain to resolve with DoH: ").strip()
query_doh(domain)
```

Output:



Optimized Performance: Created a custom DNS caching mechanism to reduce resolution latency.

DNS caching: is the process of temporarily storing DNS records to improve DNS resolutions. Below you'll find the code for caching DNS records

Script:

```
import dns.resolver
```

```
import time


class DNSCache:
    def __init__(self):
        self.cache = {}  # {domain: (response, expiration_time)}


    def query(self, domain, ttl=60):
        current_time = time.time()


        # Check cache
        if domain in self.cache:
            response, expiration_time = self.cache[domain]
            if current_time < expiration_time:
                print(f"Cache hit for {domain}")
                return response
            else:
                print(f"Cache expired for {domain}")
                del self.cache[domain]


        # Perform DNS query
        try:
            print(f"Performing DNS query for {domain}")
            resolver = dns.resolver.Resolver()
            answer = resolver.resolve(domain, 'A')
            response = [record.to_text() for record in answer]


            # Cache the response
            expiration_time = current_time + ttl
            self.cache[domain] = (response, expiration_time)
            return response
```

```
        except Exception as e:
            print(f"An error occurred: {e}")


# Example usage
dns_cache = DNSCache()
domain = input("Enter the domain to resolve with caching: ").strip()
ttl = int(input("Enter cache TTL (in seconds): ").strip())
response = dns_cache.query(domain, ttl=ttl)
if response:
    print(f"Resolved IP(s) for {domain}: {response}")
```

Output:

```
C:\Users\pilus\OneDrive\Documents\School Projects>python DNS_Caching.py
Enter the domain to resolve with caching: google.com
Enter cache TTL (in seconds): 90
Performing DNS query for google.com
Resolved IP(s) for google.com: ['142.251.47.174']
```

Concurrent TCP File Management:

Development of a multi-threaded TCP server capable of handling simultaneous client requests for file uploads and downloads.

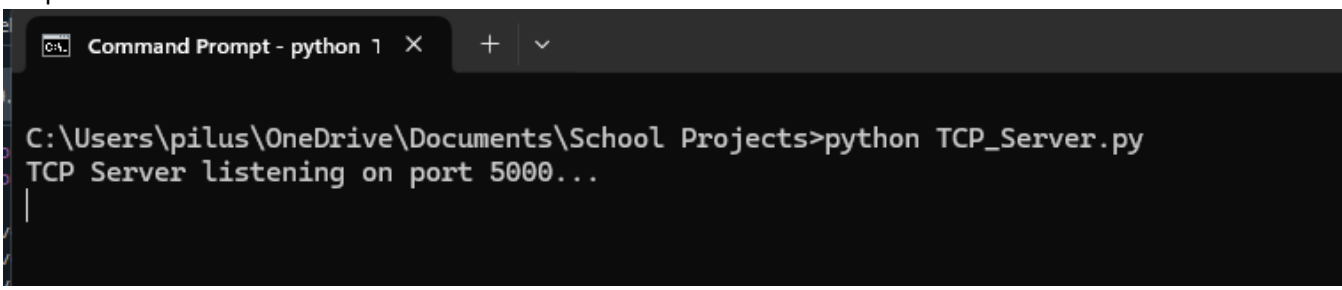
Concurrency: Uses Python's threading library to manage multiple connections without blocking.

Script:

```
import socket
import threading
import os

# Server Configuration
server_address = ('0.0.0.0', 5000)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(server_address)
server_socket.listen(5)  # Allows up to 5 simultaneous connections
print("TCP Server listening on port 5000...")

# Function to handle client requests
def handle_client(client_socket):
    with client_socket:  # Automatically closes the socket after the block
        try:
            # Receive command from client (UPLOAD or DOWNLOAD)
            command = client_socket.recv(1024).decode()
            print(f"Command received: {command}")

            if command == 'UPLOAD':
                # Handle file upload from client
                filename = client_socket.recv(1024).decode()  # Receive filename
                print(f"Receiving file: {filename}")
```

```
            with open(filename, 'wb') as f:
                while True:
                    chunk = client_socket.recv(4096)
                    if not chunk:  # No more data
                        break
                    f.write(chunk)
            print(f"File '{filename}' successfully uploaded.")

            # Send acknowledgment to the client
            client_socket.send(b"File uploaded successfully.")

        elif command == 'DOWNLOAD':
            # Handle file download request
            filename = client_socket.recv(1024).decode()  # Receive requested filename
            print(f"Client requested file: {filename}")

            if os.path.exists(filename):
                client_socket.send(b"FILE_FOUND")  # Notify client file exists
                with open(filename, 'rb') as f:
                    while chunk := f.read(4096):  # Read file in chunks
                        client_socket.send(chunk)
                print(f"File '{filename}' successfully sent to client.")
            else:
                client_socket.send(b"FILE_NOT_FOUND")  # Notify client file doesn't exist
                print(f"File '{filename}' not found.")

        else:
            print("Unknown command received.")

    except Exception as e:
        print(f"An error occurred: {e}")

# Accept clients and handle them in separate threads
while True:
    client_socket, client_address = server_socket.accept()
    print(f"Connection established with {client_address}")
    threading.Thread(target=handle_client, args=(client_socket,)).start()
```

output:



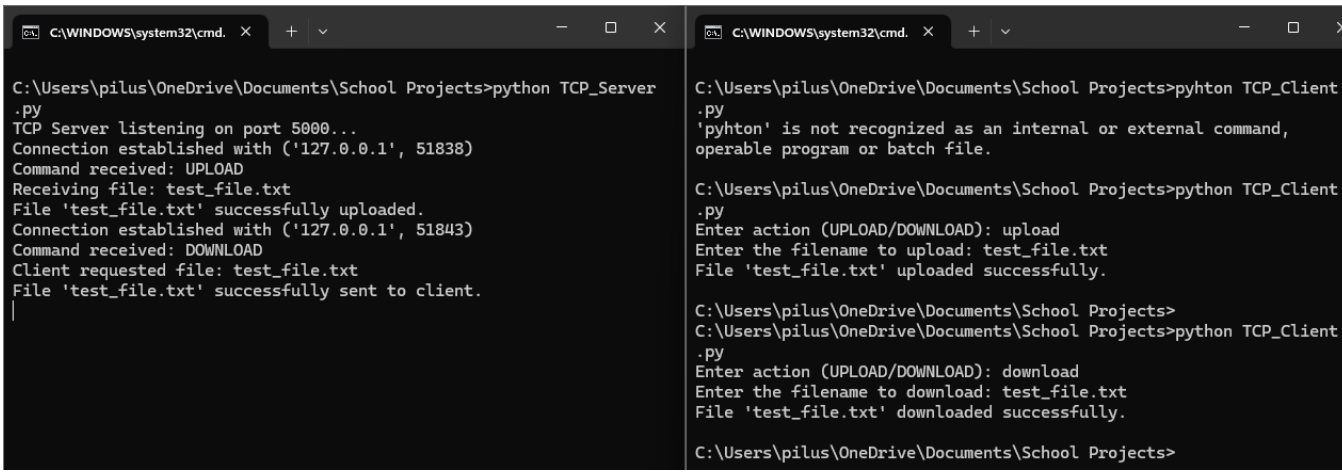
Reliability: Implemented chunk-based data transfer (4096 bytes) to ensure efficient handling of larger files.

Script:

```
import socket

# Server configuration
server_address = ('localhost', 5000)
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(server_address)

# Function to upload a file to the server
def upload_file():
    filename = input("Enter the filename to upload: ").strip()  # Prompt user for filename
    try:
        client_socket.send(b'UPLOAD')  # Send upload command
        client_socket.send(filename.encode())  # Send filename

        # Open the specified file and send its content
        with open(filename, 'rb') as f:
            while chunk := f.read(4096):  # Read file in chunks
                client_socket.send(chunk)
        print(f"File '{filename}' uploaded successfully.")
    except FileNotFoundError:
        print(f"File '{filename}' not found. Please check the filename and try again.")
    except Exception as e:
        print(f"An error occurred during file upload: {e}")

# Function to download a file from the server
def download_file():
    filename = input("Enter the filename to download: ").strip()  # Prompt user for filename
    try:
        client_socket.send(b'DOWNLOAD')  # Send download command
        client_socket.send(filename.encode())  # Send filename

        # Save the received file with a prefix
        with open(f"downloaded_{filename}", 'wb') as f:
            while True:
                chunk = client_socket.recv(4096)  # Read in chunks
                if not chunk:  # No more data
                    break
                f.write(chunk)
        print(f"File '{filename}' downloaded successfully.")
    except Exception as e:
        print(f"An error occurred during file download: {e}")
```

```
# Main logic to select upload or download
action = input("Enter action (UPLOAD/DOWNLOAD): ").strip().upper()
if action == 'UPLOAD':
    upload_file()
elif action == 'DOWNLOAD':
    download_file()
else:
    print("Invalid action. Please enter 'UPLOAD' or 'DOWNLOAD'.")

client_socket.close()
```

Output:



Real-time UDP Communication

A lightweight broadcast chat application built on the User Datagram Protocol (UDP).

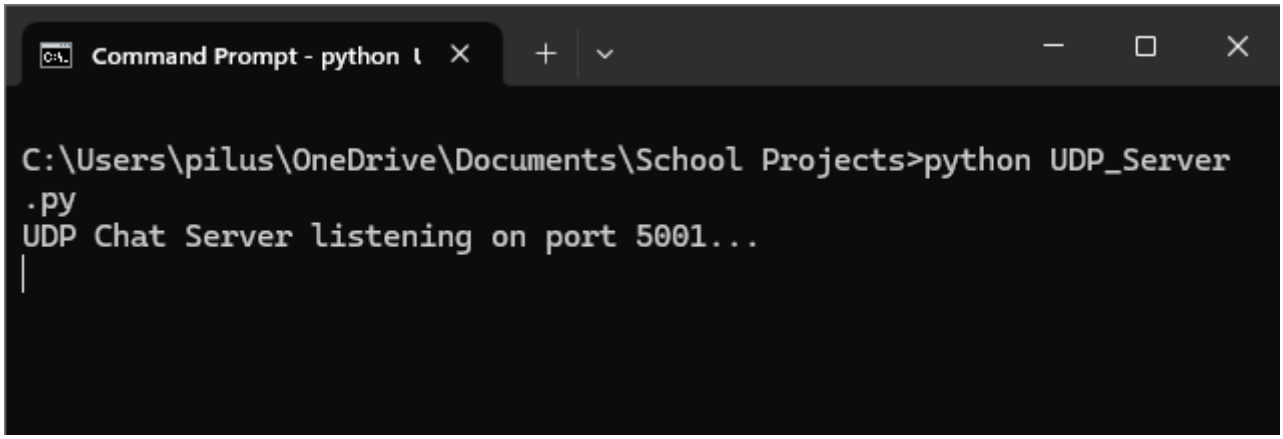
Client Management: The server maintains a dynamic set of active client addresses to facilitate real-time broadcasting

Script:

```
import socket


# Define server address and port

server_address = ('0.0.0.0', 5001)  # Listens on all available network interfaces

clients = set()  # Store connected client addresses


# Create a UDP socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server_socket.bind(server_address)

print("UDP Chat Server listening on port 5001...")
```

```
#Shutdown the server
def close_server():
    """Gracefully shuts down the server."""
    print("\nShutting down the server...")
    server_socket.close()
    print("Server closed successfully.")
    exit()
try:
    while True:
        # Receive message from a client
        message, client_address = server_socket.recvfrom(1024)

        # If a new client joins, add them to the list
        if client_address not in clients:
            clients.add(client_address)

        print(f"Received from {client_address}: {message.decode()}")

        # Broadcast the message to all clients except the sender
        for client in clients:
            if client != client_address:
                server_socket.sendto(message, client)
except KeyboardInterrupt():
    close_server()
```

output:



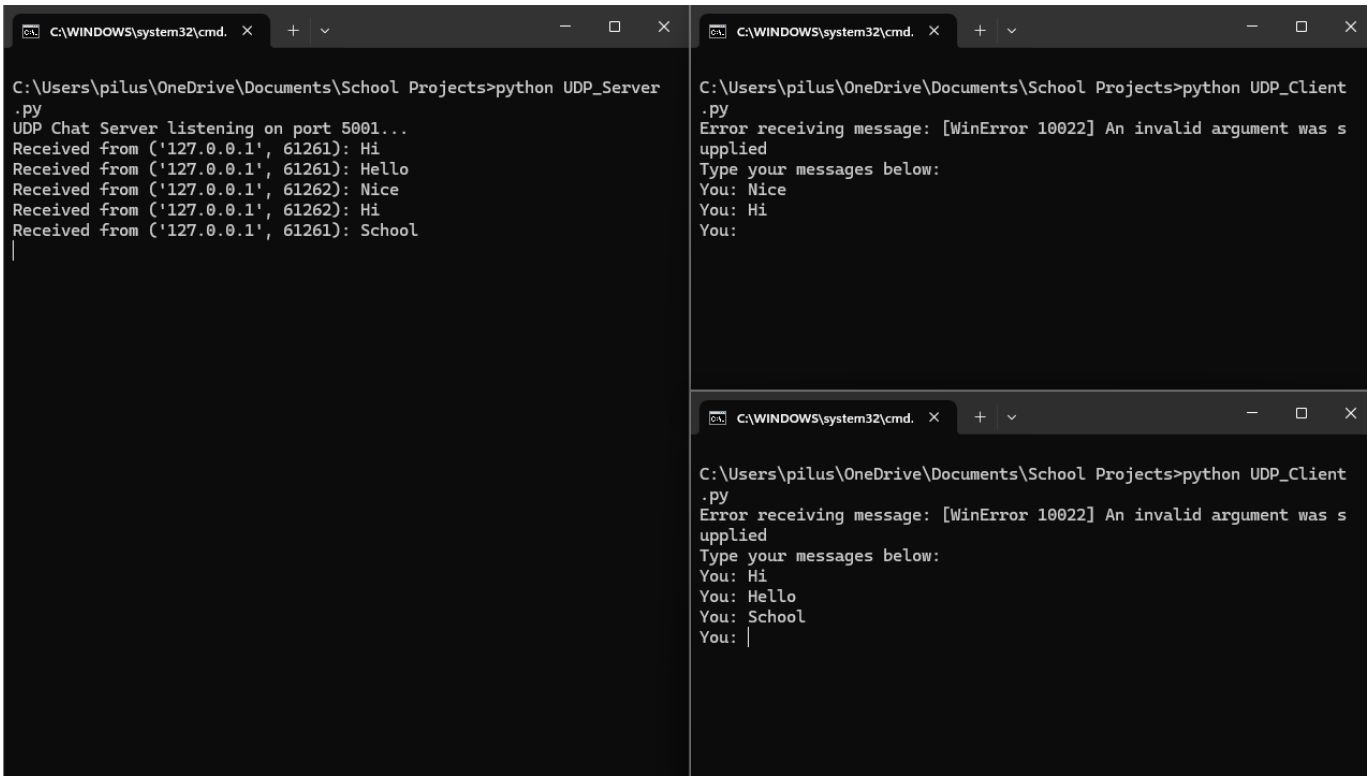
Asynchronous Messaging: Employs background threads on the client-side to listen for incoming messages while allowing the user to type concurrently.

Script:

```
import socket
import threading

# Define the server's address and port
server_address = ('localhost', 5001)  # Server's address

# Create a UDP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

def receive_messages():
    """Continuously receive and display messages from the server."""
    while True:
        try:
            message, _ = client_socket.recvfrom(1024)  # Receive message from server
            print("\nReceived:", message.decode())  # Display received message
        except Exception as e:
            print(f"Error receiving message: {e}")
            break

# Start a thread to continuously listen for messages from the server
threading.Thread(target=receive_messages, daemon=True).start()

print("Type your messages below:")

while True:
    try:
        message = input("You: ").strip()  # Get user input and remove extra spaces
        if not message:  # Avoid sending empty messages
            print("Message cannot be empty. Please enter a valid message.")
            continue
        client_socket.sendto(message.encode(), server_address)  # Send the message to the server
    except KeyboardInterrupt:
        print("\nExiting chat...")
```

```
        break
    except Exception as e:
        print(f"Error sending message: {e}")
        break
```

Output:



Works Cited

Atkin, R., 2022. *Using multi-threaded Python for large scale DNS name resolution.* [Online]
Available at: https://richardatkin.com/post/2022/01/15/Using-multithreaded-Python-to-speed-up-DNS-name-resolution.html
[Accessed 8 March 2025].

Bertram, A., 2020. *How to Use OpenSSL to Generate Certificates.* [Online]
Available at: https://www.progress.com/blogs/how-to-use-openssl-to-generate-certificates
[Accessed 1 March 2025].

Bob Halley, 2021. *dnspython.* [Online]
Available at: https://dnspython.readthedocs.io/en/latest/index.html
[Accessed 13 March 2025].

forbiddenlake, 2023. *Crafting a DNS-over-HTTPS GET request manually.* [Online]
Available at: https://blog.kiserai.net/crafting-a-dns-over-https-get-request-manually
[Accessed 19 March 2025].

GeeksforGeeks, 2024. *DNSSEC (Domain Name System Security Extensions) Implementation.* [Online]
Available at: https://www.geeksforgeeks.org/dnssec-domain-name-system-security-extensions-implementation/
[Accessed 10 March 2025].

GeeksforGeeks, 2024. *Transport Layer Security (TLS).* [Online]
Available at: https://www.geeksforgeeks.org/transport-layer-security-tls/
[Accessed 1 March 2025].

GeekstoGeeks, 2023. *DNS Load Balancing: Round Robin, Global Server Load Balancing.* [Online]
Available at: https://www.geeksforgeeks.org/dns-load-balancing-round-robin-global-server-load-balancing/
[Accessed 19 March 2025].

Joshua, C., 2023. *What is transport layer security and what does it do?.* [Online]
Available at: https://www.avast.com/c-what-is-transport-layer-security
[Accessed 2 March 2025].

Metzler, S., 2024. *TLS Encryption: What Is It & Why it's Important.* [Online]
Available at: https://www.securew2.com/blog/what-is-tls-encryption
[Accessed 2 March 2025].

Ranke, P., 2023. *Mutual TLS: A Tutorial.* [Online]
Available at: https://builtin.com/software-engineering-perspectives/mutual-tls-tutorial
[Accessed 5 March 2025].

Schrader, D., 2025. *What DNS over HTTPS (DoH) Is and How to Enable in Windows 10.* [Online]
Available at: What DNS over HTTPS (DoH) Is and How to Enable in Windows 10
[Accessed 19 March 2925].